Introduction to Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI and Rational Agents **2**
- 1.1 Agents and their environments . . . 2
- 1.2 Reflex and planning agents . . . 2
- 1.3 Specifying a task with PEAS . . . 3
- 1.4 Properties of the environment . . . 3

2 Search: State Spaces and Uninformed Search **4**
- 2.1 Formulating a search problem . . . 4
- 2.2 State-space graphs and search trees . . . 5
- 2.3 A common tree-search procedure . . . 6
- 2.4 Depth-first search . . . 6
- 2.5 Breadth-first search . . . 7
- 2.6 Iterative deepening . . . 7
- 2.7 Uniform-cost search . . . 8

Lecture 1: Introduction to AI and Rational Agents

1.1 Agents and their environments

An AGENT observes an ENVIRONMENT through sensors and changes it through actuators. Its observations inform the actions it takes. The environment together with its agents forms the WORLD.

A RATIONAL AGENT chooses actions to obtain the best expected outcome according to its goals or preferences. The criterion concerns what can be expected from the available information, rather than a guarantee that every realized outcome will be favorable.

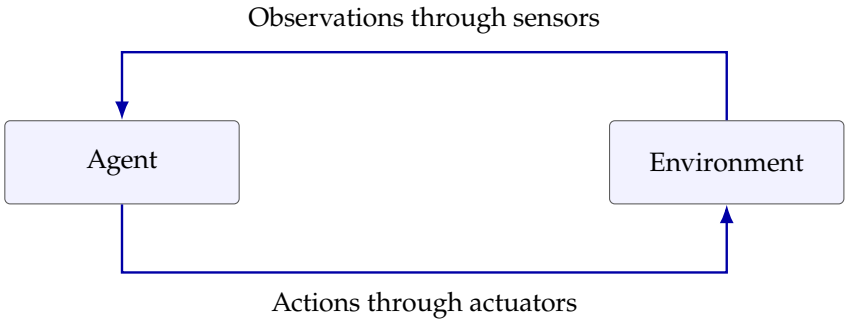


Figure 1: The interaction between an agent and its environment. Sensors supply observations; actuators carry out actions.

For a checkers-playing agent, the environment is the board and opposing player, and an action is a legal piece move. The agent's preferences determine which outcomes it should seek.

1.2 Reflex and planning agents

A REFLEX AGENT selects an action from its current percept, possibly using memory or an estimate of the current state. It does not compare the future consequences of candidate actions.

A PLANNING AGENT uses a model of how actions change the world to reason about possible futures. It predicts the consequences of actions and chooses among them according to its objective. The distinction is between reacting to the present and explicitly looking ahead; retaining memory alone does not make an agent a planner.

1.3 Specifying a task with PEAS

A PEAS DESCRIPTION fixes four components of the task:

Component	What it specifies
Performance measure	How outcomes are evaluated.
Environment	Where the agent acts and what affects it.
Actuators	How the agent changes the environment.
Sensors	How the agent obtains information.

The performance measure supplies the objective; the remaining components describe the circumstances and means of achieving it.

1.4 Properties of the environment

The information available to an agent and the effects of its actions determine what its design must handle.

- FULLY OBSERVABLE versus PARTIALLY OBSERVABLE: does the agent observe the relevant state completely? With partial observations, it must maintain an internal estimate of the state.
- DETERMINISTIC versus STOCHASTIC: does a state and action determine one next state, or a distribution of possible next states? Stochastic transitions require reasoning about uncertain outcomes.
- SINGLE-AGENT versus MULTI-AGENT: do other agents also act in the environment? Their behavior affects the outcome, and randomizing actions can prevent predictability.
- STATIC versus DYNAMIC: can the environment change while the agent is deciding what to do? In a dynamic environment, deliberation may leave observations outdated.
- KNOWN versus UNKNOWN dynamics: is the transition model available to the agent? Unknown dynamics require learning how actions affect the world, potentially through deliberate exploration.

Knowing the dynamics does not remove randomness: an agent can know a stochastic transition model while remaining uncertain about the next outcome.

Source. Nikhil Sharma, *CS 188: Introduction to Artificial Intelligence*, Spring 2024, Note 1: Agents.

Lecture 2: Search: State Spaces and Uninformed Search

Planning requires a model of how actions change the world and a test for whether a goal has been reached. A complete planning method finds a plan whenever one exists; an optimal method finds a plan of minimum cost. An agent can plan once or replan as new observations arrive. A reflex rule can also be rational when its chosen action already maximizes the relevant objective.

2.1 Formulating a search problem

Definition 2.1 [Search problem]. A search problem consists of a state space $\mathcal{S}$, available actions $A(s)$, a transition model $T(s, a)$, action costs $c(s, a)$, a start state s_0, and a goal test Goal(s). A plan is a sequence of actions that takes s_0 to a goal state. Its cost is the sum of its action costs.

The successor function combines the actions, their resulting states, and their costs. In route planning through Romania, states are cities and successors are adjacent cities, with road distances as costs. The start is Arad and the goal test asks whether the current city is Bucharest.

A world state describes the full environment. A search state retains the information needed for the chosen planning problem. This abstraction depends on the objective.

	Pathing	Eat-all-dots
State	Position (x, y)	Position and remaining-dot booleans
Actions	Legal north, south, east, west moves	The same legal moves
Transition	Update position	Update position and consumed dots
Goal test	Reach the destination	No dots remain

For 120 agent positions, four orientations, two ghosts with 12 positions each, and 30 independently represented dot booleans, the full encoding has

$$120 \cdot 4 \cdot 12^2 \cdot 2^{30}$$

possible configurations. The pathing abstraction has only 120 states; eat-all-dots has $120 \cdot 2^{30}$. These are counts of combinations in the encoding; restrictions on reachability can reduce the reachable subset.

To eat all dots while keeping ghosts continuously scared, the search state must also retain the remaining power pellets and the remaining scared time. Information can be omitted only when it is irrelevant to future decisions.

2.2 State-space graphs and search trees

A STATE-SPACE GRAPH has one vertex per state and directed edges for action results. A SEARCH TREE instead unfolds possible plans from the start state: its nodes represent paths, and their labels give the states reached by those paths. Different paths can therefore create different tree nodes with the same state label.

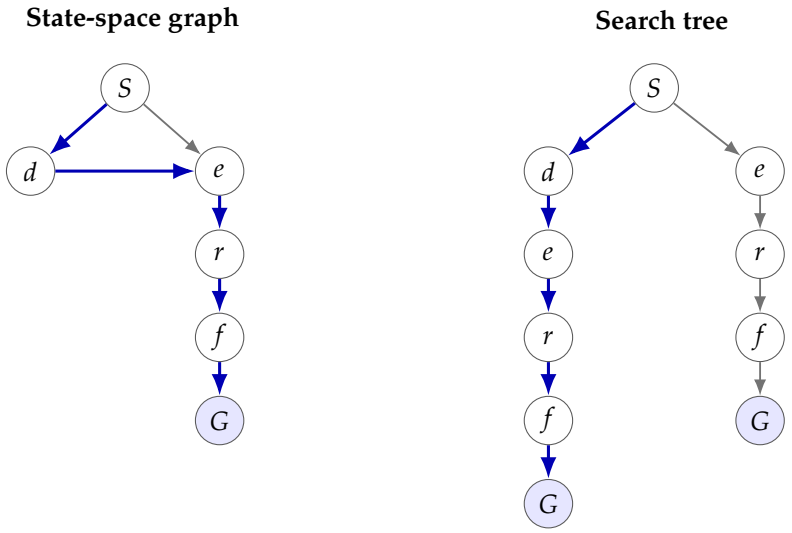


Figure 2: Selected transitions and their search tree. The state e occurs once in the graph but twice in the tree, reached by $S \to e$ and $S \to d \to e$. Blue marks $S \to d \to e \to r \to f \to G$.

Cycles in a finite graph can produce an infinite search tree. Neither structure needs to be constructed in full: generate successors only when a node is expanded, and retain the partial plans currently under consideration.

2.3 A common tree-search procedure

The FRONTIER, or FRINGE, contains generated nodes that have not yet been selected for exploration. A node records its state, parent, incoming action, and cumulative path cost g.

Algorithm 2.1: Tree search

Input: Search problem and an initially empty frontier F with a chosen removal rule.
Output: A plan reaching a goal, or failure if the frontier is exhausted.

Insert the root node $(s_0, \varnothing, \varnothing, 0)$ into F
while F is not empty **do**
    Remove a node n from F according to the removal rule
    if Goal(n.state) **then**
        return the plan obtained by following n's parent links
    for each $a \in A(n.\text{state})$ **do**
        $s' \leftarrow T(n.\text{state}, a)$
        $g' \leftarrow n.g + c(n.\text{state}, a)$
        Insert (s', n, a, g') into F
return failure

The removal rule determines the exploration strategy. This is tree search: it does not suppress repeated states. In particular, the goal test occurs when a node is removed, which matters for cost-sensitive search.

UNINFORMED SEARCH uses the problem's transitions and costs without an estimate of how close a state is to a goal. To compare methods, let $b > 1$ bound the branching factor, m be the maximum tree depth, and s be the shallowest goal depth. A full tree through depth m has

$$1 + b + b^2 + \cdots + b^m = O(b^m)$$

nodes. Time bounds below count tree-node operations, apart from the additional cost of implementing a priority queue.

2.4 Depth-first search

DEPTH-FIRST SEARCH (DFS) removes a deepest frontier node first, using a last-in, first-out STACK. Newly generated children are explored before older siblings.

With finite maximum depth m, DFS may explore the entire tree, taking $O(b^m)$ time. It retains the current branch and pending siblings along it, requiring $O(bm)$ space. It can fail to return when a branch is infinite, even if another branch contains a goal. Thus unrestricted tree-search DFS is not complete. It is also not optimal: the first goal in its exploration order need not be shallow or inexpensive.

2.5 Breadth-first search

Breadth-first search (BFS) removes a shallowest frontier node first, using a first-in, first-out queue. It finishes each depth before exploring the next one.

For finite b and a goal at finite depth s, BFS is complete and finds a shallowest goal. This is a least-cost plan when all action costs are equal and positive; unequal costs break the connection between depth and cost.

There are $1 + b + \cdots + b^s = O(b^s)$ nodes through the goal depth. With the goal test on removal, other nodes at depth s can generate children at depth $s + 1$ before a goal is selected. A bound on generated nodes and frontier storage is therefore $O(b^{s+1})$. The exponential frontier is BFS's main space limitation.

DFS can reach a deep goal quickly if its early branches are favorable; BFS is useful when a shallow solution exists or infinite branches would trap DFS. Their performance depends on the tree and successor order.

2.6 Iterative deepening

Iterative deepening search repeatedly runs depth-limited DFS, increasing the limit until a goal is found. A limit of zero checks only the start; subsequent runs allow depths $1, 2, \ldots$.

Although upper levels are revisited, the deepest level dominates for $b > 1$. To reach depth s, the work is bounded by

$$\sum_{i=0}^{s}(s - i + 1)b^i = b^s \sum_{j=0}^{s} \frac{j + 1}{b^j} = O(b^s).$$

Each run stores only its DFS frontier, giving $O(bs)$ space at the final limit. Finite branching makes it complete, and equal positive action costs make its first solution optimal.

For $b = 10$ and $s = 5$, counting full levels and omitting the root,

$$\begin{aligned} \text{BFS:} \quad & 10 + 100 + 1000 + 10000 + 100000 = 111110, \\ \text{iterative deepening:} \quad & 5 \cdot 10 + 4 \cdot 100 + 3 \cdot 1000 + 2 \cdot 10000 + 100000 = 123450. \end{aligned}$$

The repeated work is modest compared with the storage saved.

2.7 Uniform-cost search

Uniform-cost search (UCS) removes a node with minimum cumulative path cost $g(n)$, using a priority queue. It explores cost contours rather than depth levels. With equal positive action costs, this ordering agrees with BFS's depth ordering, up to ties.

Proposition 2.2 [UCS optimality]. *With nonnegative action costs, a goal returned by UCS has minimum plan cost.*

Proof. Suppose UCS removes a goal of cost C, but a cheaper goal path exists. There is still a frontier node on that path: initially its root is present, and expanding a prefix inserts its next prefix. Nonnegative costs make every prefix of the cheaper path cost at most its final cost, which is less than C. UCS would remove such a prefix before the goal of cost C, a contradiction. □

Optimality on return does not alone guarantee termination. If every action cost is at least $\epsilon > 0$, branching is finite, and an optimal plan has finite cost C^*, only finitely many tree nodes can precede that plan. UCS is then complete. A positive lower bound rules out endlessly exploring ever-longer paths whose costs stay below the goal cost.

Nodes expanded before termination have cost at most C^* and depth at most $d = \lfloor C^*/\epsilon \rfloor$. There are $O(b^d)$ such nodes; their generated children give time and space bounds $O(b^{d+1})$. Thus the effective depth is set by C^*/ϵ. Maintaining a heap adds a logarithmic factor per priority-queue operation.

UCS uses no information about the goal's location, so it can explore many cheap paths in unhelpful directions. DFS, BFS, and UCS share the same tree-search procedure; their frontier removal rules account for their different behavior.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Spring 2024, Lecture 2: Search; Nikhil Sharma, Note 2.